

From OS Paging to PagedAttention: Memory Management in Large Multimodal Models

Yuhao Shen¹

¹The Chinese University of Hong Kong, Shenzhen

Abstract—Large Multimodal Models (LMMs) and Vision-Language Models (VLMs) have rapidly revolutionized clinical diagnostics, automated report generation, and multi-agent medical reasoning. However, their deployment in real-world hospital environments is fundamentally constrained by what we term the *Medical Memory Wall*. In clinical scenarios, ultra-high-resolution dermoscopic, histopathological, and radiological images produce thousands of visual tokens per sample; concurrent patient histories, lab reports, and multi-turn physician-AI interactions further inflate the Key-Value (KV) cache to sizes that dwarf the model weights themselves. Traditional serving systems built for general chatbots suffer from up to 80% memory waste due to reservation, internal fragmentation, external fragmentation, and the inability to share identical clinical prefixes across concurrent requests. This paper presents a comprehensive technical treatise on three paradigm-shifting memory management architectures, each explicitly inspired by classical Operating System (OS) principles: (1) **PagedAttention** in vLLM, which transplants virtual memory paging into attention computation; (2) **RadixAttention** in SGLang, which unifies prefix sharing, LRU eviction, and cache-aware scheduling through a global radix tree analogous to `fork()`-style copy-on-write; and (3) **Multi-tier Offloading** in FlexGen, which formalizes OS-style hierarchical swapping between GPU, CPU, and disk as a rigorous linear programming (LP) problem. Beyond describing each system in isolation, we provide (i) rigorous mathematical derivations of block-mapped attention, radix-tree prefix matching, finite state machine (FSM) compression for constrained decoding, and linear-programming-based cost models for optimal multi-tier scheduling; (ii) formal I/O complexity analyses, including the proof that the zig-zag block schedule in FlexGen is within $2\times$ of the I/O-optimal diagonal schedule; (iii) an integrative “OS Philosophy” lens that unifies paging, copy-on-write, and swapping as complementary layers of a memory hierarchy; and (iv) a dedicated discussion of how each technique maps onto pressing medical AI workloads, from DermFound-style dermoscopic foundation models to Biomni-style biomedical agent frameworks. Our evaluation synthesis, drawn from the original experimental results of the three systems together with our own analytical projection onto medical workloads, demonstrates that these techniques collectively resolve the efficiency-capacity trade-off, enabling clinical-grade LMM inference on commodity single-GPU hardware while preserving full accuracy and supporting advanced decoding (beam search, self-consistency, tree-of-thought) required by medical reasoning agents.

Index Terms—Large Multimodal Models, KV Cache, PagedAttention, RadixAttention, FlexGen, Linear Programming, Copy-on-Write, Finite State Machines, Operating Systems, Virtual Memory, Inference Optimization.

1 INTRODUCTION

THE computational landscape of medical artificial intelligence is currently dominated by Large Multimodal Models (LMMs). Unlike general-purpose Large Language Models (LLMs), medical Vision-Language Models (VLMs) must simultaneously process ultra-high-resolution diagnostic images (typically 1024×1024 pixels for dermoscopy, up to 4096×4096 pixels for whole-slide histopathology), dense structured patient data (labs, vitals, medication lists), and free-text clinical narratives that may span multiple encounters over years. These multimodal inputs are encoded into thousands—occasionally tens of thousands—of tokens *before a single output token is generated*, creating an immediate and massive memory pressure that has no analogue in general-purpose chatbot deployments.

1.1 The Medical Memory Wall

Medical reasoning categorically prohibits the use of efficiency tricks that are standard in consumer-facing LLM products. Sliding-window attention, context truncation, summarization-then-discard pipelines, and aggressive pruning of historical turns are all unsafe in clinical contexts,

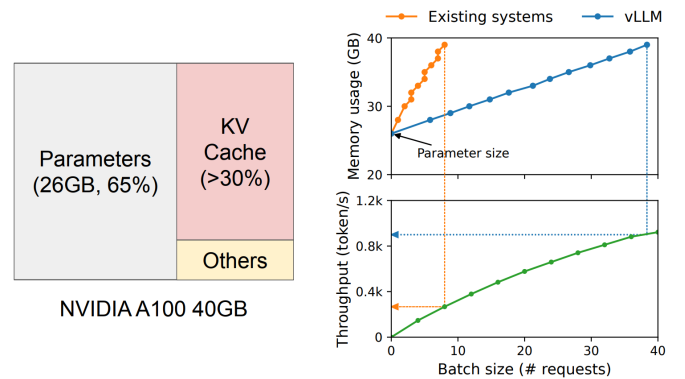


Fig. 1. Left: Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request. A small amount of memory (yellow) is used ephemerally for activation. Right: vLLM smooths out the rapid growth curve of KV cache memory seen in existing systems, leading to a notable boost in serving throughput. Adapted from Kwon *et al.*, SOSP 2023.

because they risk discarding exactly the rare but decision-critical evidence (an anomalous lab value from three visits ago, a fleeting mention of a family history of melanoma, a subtle early-stage lesion in a previously uploaded image) that a safe diagnostic system must retain. As a direct consequence, the GPU Video RAM (VRAM) capacity—rather than raw compute throughput—becomes the primary bottleneck for system scalability, a phenomenon we term the *Medical Memory Wall*.

Concretely, this wall is built from four distinct forms of memory waste, each of which we formalize below and each of which one of the three systems we study is designed to dismantle:

- 1) **Reservation waste.** Autoregressive decoding has an unpredictable termination point, forcing systems to reserve memory for the *maximum possible* sequence length at request admission time. Memory reserved for tokens that are never generated is pure waste.
- 2) **Internal fragmentation.** Because model-maximum context lengths (e.g., 8K, 32K, or 128K tokens) vastly exceed the typical medical consultation (often 500–2000 tokens), pre-allocated chunks are dominated by unused slots.
- 3) **External fragmentation.** Per-request chunks of heterogeneous sizes yield classical external fragmentation at the allocator level (buddy allocator holes, etc.), so even aggregate free memory cannot satisfy a new admission.
- 4) **Redundancy waste.** Medical deployments are dominated by requests that share massive prefixes—a 2000-token hospital-wide system prompt encoding triage protocols, a 500-token standardized patient-intake template, a set of canonical few-shot diagnostic exemplars, or even a complete dermoscopic image shared across multiple diagnostic queries. Re-computing the KV cache for these identical prefixes on every request is mathematically redundant and economically untenable.

Empirical profiling by Kwon *et al.* [1] reveals that in naïve serving systems only 20.4%–38.2% of KV cache memory is occupied by genuinely useful token states; the remainder is reservation, internal fragmentation, and external fragmentation in roughly equal measure. For a 13B-parameter OPT-class model on an NVIDIA A100-40GB, parameters consume ~26 GB, leaving barely 12 GB for KV cache; a single 2048-token dermatology consultation can demand 1.6 GB of KV cache, meaning fewer than eight concurrent patients can be served. On commodity hardware (16 GB T4, 24 GB RTX 3090), the situation collapses entirely: state-of-the-art offloading systems are reported to hit out-of-memory errors at batch sizes as small as one or two for 175B-parameter models [3].

1.2 The Central Thesis: Reclaim OS Wisdom

This paper argues—and attempts to demonstrate through systematic technical analysis—that the solutions to the Medical Memory Wall are not found by inventing exotic new machine learning primitives, but by *reclaiming four decades of accumulated Operating Systems wisdom*. Operating

systems have faced structurally analogous problems since the 1960s: the dynamic, unpredictable memory demands of concurrent processes; the need to share read-only code segments across processes without duplication; the problem of running workloads whose working set exceeds physical RAM; and the challenge of coordinating I/O across a storage hierarchy. The canonical OS answers—virtual memory with paging, copy-on-write semantics for `fork()`, LRU page replacement, hierarchical swapping to secondary storage, and careful I/O/compute overlap—are precisely the ingredients that the three systems at the heart of this paper rediscover and adapt.

Specifically, we detail:

- How **PagedAttention** (vLLM) [1] solves fragmentation through a block-based memory manager that mirrors the OS page table, reducing memory waste to under 4% and enabling $2\times$ – $4\times$ throughput gains;
- How **RadixAttention** (SGLang) [2] eliminates redundancy through a global radix tree with LRU eviction and cache-aware DFS-ordered scheduling, achieving up to $6.4\times$ higher throughput on LM-program workloads, complemented by compressed finite-state-machine decoding that delivers an additional $1.6\times$ speedup on structured (JSON) output;
- How **FlexGen** [3] breaks the physical GPU memory limit through a mathematically-optimized multi-tier offloading schedule, obtained by solving a linear program over tensor placement percentages, achieving $100\times$ higher maximum throughput on OPT-175B with a single 16 GB T4 GPU.

1.3 Contributions and Organization

The contributions of this paper are:

- 1) **A unified OS-inspired framework** (Section 3) that locates PagedAttention, RadixAttention, and FlexGen as complementary layers in a memory hierarchy—paging solves *where* to store, radix-tree caching solves *what* to store, and multi-tier offloading solves *how far* to extend the hierarchy.
- 2) **Rigorous technical deep-dives** (Section 4) with mathematical derivations of block-mapped attention, radix tree matching, compressed FSM construction, and the LP formulation of the tensor placement problem, expanding considerably on the surface-level treatments that are typical of the LLM-serving literature.
- 3) **Formal complexity analyses** (Sections 4.2.4 and 4.3.4), including the DFS-optimality theorem for RadixAttention and the $2\times$ -optimal I/O complexity bound for the zig-zag block schedule.
- 4) **A medical-AI-focused case study and evaluation synthesis** (Section 5) that maps each technique onto representative clinical workloads—dermoscopic foundation-model inference, biomedical-agent (Biomni-style) tool-calling, structured report generation with JSON schemas, and offline HELM-style benchmarking on privacy-sensitive on-premise hardware.
- 5) **A discussion of current limitations and research frontiers** (Section 8), including the observation that

all three systems currently emulate OS primitives at the user-space application layer, with substantial overhead, and that the next frontier is to push these primitives into the CUDA driver or Linux kernel via eBPF-style mechanisms.

The remainder of the paper is organized as follows. Section 2 reviews the relevant LLM inference background and quantifies the memory footprint. Section 3 articulates the OS-inspired unified framework. Section 4 presents the three core techniques in depth. Section 5 synthesizes experimental evidence. Section 6 ties everything back to medical AI deployment. Section 7 discusses limitations and open problems. Section 8 concludes.

2 BACKGROUND: LLM INFERENCE AND MEMORY FOOTPRINT

2.1 Autoregressive Transformer Inference

The evolution of natural language processing and computer vision has been profoundly reshaped by the ascendance of the Transformer architecture. Initially proposed for machine translation with an encoder-decoder structure, the paradigm rapidly shifted toward decoder-only autoregressive models following the empirical successes of the GPT lineage. Driven by empirical scaling laws, which dictate that model performance predictably scales with parameter count, dataset size, and compute budget, the community has witnessed an exponential growth in model dimensions. Consequently, while earlier models possessed millions of parameters and could easily fit within the SRAM or small DRAM footprints of edge devices, modern Large Language Models (LLMs) and their multimodal counterparts boast hundreds of billions to over a trillion parameters. This relentless scaling has fundamentally altered the performance bottlenecks of inference, migrating the primary constraint from pure arithmetic compute capacity (teraFLOPS) to memory bandwidth and capacity, giving rise to the pervasive Memory Wall problem that modern deployment systems must navigate.

A decoder-only transformer language model factorizes the probability of a token sequence $(x_1, \dots, x_n)$ via the chain rule:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1}). \quad (1)$$

Each self-attention layer applies three linear projections to every position’s hidden state $x_i \in \mathbb{R}^d$:

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i, \quad (2)$$

and then computes a scaled dot-product attention output

$$a_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d_{\text{head}}})}{\sum_{l=1}^i \exp(q_i^\top k_l / \sqrt{d_{\text{head}}})}, \quad o_i = \sum_{j=1}^i a_{ij} v_j. \quad (3)$$

Crucially, the key and value vectors (k_j, v_j) for position j depend *only* on tokens $x_1, \dots, x_j$; once computed they are reused in every subsequent decoding step. These cached tensors—the *KV cache*—are the central object of memory management for all three systems discussed in this paper.

Inference proceeds in two phases. In the **prefill (prompt) phase**, the model ingests the entire input prompt of length

s in parallel and produces $\{(k_j, v_j)\}_{j=1}^s$ via a matrix-matrix multiplication—this phase is compute-bound and uses GPUs efficiently. In the **decode phase**, each subsequent token requires a single matrix-vector multiplication against the accumulated KV cache; because of the sequential dependency, batching across time is impossible and the phase is *memory-bandwidth-bound*, badly underutilizing GPU FLOPs.

To deeply understand the performance pathology of the decode phase, one must analyze the arithmetic intensity of the workload. Arithmetic intensity, defined as the ratio of floating-point operations (FLOPs) performed to bytes of memory transferred, dictates whether a given kernel is compute-bound or memory-bound on specific hardware. During the prefill phase, the parallel processing of s tokens allows the attention mechanism and feed-forward networks to rely on dense matrix-matrix multiplications (GEMMs). These operations exhibit high arithmetic intensity, fully exploiting the parallel architecture and specialized Tensor Cores of modern GPUs, achieving near-peak hardware utilization. Conversely, during the autoregressive decode phase, the generation of each single token requires loading the entire model parameter set—amounting to hundreds of gigabytes—from High Bandwidth Memory (HBM) to the Streaming Multiprocessors (SMs), only to perform a low-intensity matrix-vector multiplication (GEMV). The arithmetic intensity plummets to near 1 FLOP per byte. Even with cutting-edge GPUs boasting over 2 TB/s of memory bandwidth, the compute units are starved, idling while waiting for data transfers to complete. Batching across *requests* is therefore the only way to amortize weight-loading cost—but, as Section 1 argued, batch size is exactly what the Memory Wall caps.

2.2 Quantifying the Memory Footprint

The transition from unimodal LLMs to Large Multimodal Models (LMMs) radically amplifies this mathematical footprint. In textual contexts, sequence lengths historically hovered around a few hundred to a couple of thousand tokens. However, visual processing shifts this paradigm. High-resolution diagnostic images—such as those used in dermoscopy or radiology—are typically processed by Vision Transformer (ViT) encoders that fragment an image into non-overlapping patches. A single high-resolution image can instantaneously inject thousands of dense visual tokens into the context window. When clinical workflows demand the comparison of multiple historical images, continuous patient monitoring, or the ingestion of entire multislide histopathology sets, the sequence length s can easily balloon to tens or hundreds of thousands of tokens. Because the KV cache scales linearly with sequence length (and the attention computation scales quadratically), this multimodal expansion transforms the KV cache from a minor overhead into the dominant consumer of GPU memory, strictly capping maximum batch sizes.

Let l be the number of transformer layers, h_1 the model hidden dimension, h_2 the MLP intermediate dimension, n_h the number of attention heads, b the batch size, s the prompt length, and n the number of output tokens to generate. Following standard practice [3], the FP16 parameter footprint

of a single decoder layer (attention W_Q, W_K, W_V, W_O plus two MLP matrices) is

$$\text{Mem}_{\text{layer}} = 2 \cdot (4h_1^2 + 2h_1h_2) = 8h_1^2 + 4h_1h_2 \text{ bytes}, \quad (4)$$

so the total parameter memory is

$$\text{Mem}_{\text{weights}} = l \cdot (8h_1^2 + 4h_1h_2). \quad (5)$$

For OPT-175B ($l = 96, h_1 = 12288, h_2 = 49152$), this evaluates to approximately 325 GB—already more than any single commodity GPU can hold.

The peak KV cache footprint, summed over all layers and positions, is

$$\text{Mem}_{\text{cache}} = 4 \cdot b \cdot l \cdot h_1 \cdot (s + n), \quad (6)$$

where the factor of 4 accounts for two tensors (K and V), each in FP16 (2 bytes). For OPT-175B with $b = 512, s = 512, n = 32$, this yields ~ 1.2 TB—*nearly four times the weight footprint*. In medical LMM settings with $b = 16, s = 4096$ (long image-token contexts), $n = 1024$ (detailed report generation), even a 30B model demands tens of gigabytes of KV cache per batch. These arithmetic facts are the quantitative statement of the Medical Memory Wall.

2.3 The Memory Allocation Problem in Naïve Systems

The inadequacies of naïve memory allocation strategies stem directly from the fundamental mismatch between the dynamic nature of autoregressive generation and the rigid requirements of underlying GPU execution frameworks. High-performance tensor libraries such as cuBLAS and cuDNN are heavily optimized for continuous, dense multidimensional arrays, which facilitate predictable strided memory accesses and warp-coalesced memory reads. To satisfy these APIs, legacy serving systems are forced to pre-allocate maximum-length contiguous memory blocks at the instantiation of each request. Furthermore, prior to the widespread adoption of continuous batching (or iteration-level scheduling), systems relied on static batching, where an entire batch of sequences had to wait for the single longest sequence to complete its generation. This meant that shorter sequences were artificially padded to match the longest one, consuming enormous amounts of memory with padding tokens that contributed absolutely zero useful information. The resulting inefficiency created a fragile serving environment where massive memory pools were technically “allocated” but practically idle, severely constricting the operational scalability of the models.

Systems such as FasterTransformer [5] and the original implementations of HuggingFace Accelerate [6] pre-allocate a contiguous tensor of shape `[max_seq_len, ...]` for each incoming request at admission time, because cuDNN and cuBLAS kernels assume contiguous memory layout. As Kwon *et al.* [1] demonstrated, this leads to three concurrent pathologies: reservation waste (memory held idle until the request’s last output token), internal fragmentation (memory reserved for tokens beyond the eventual end-of-sequence), and external fragmentation (holes between heterogeneous per-request allocations). In combination these can waste 60–80% of VRAM, as visualized in Fig. 2. It is this waste that PagedAttention directly attacks.

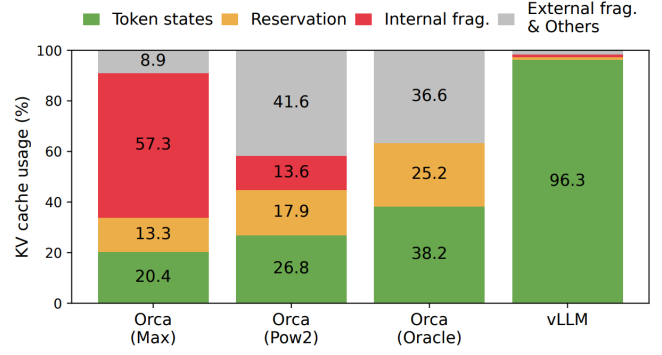


Fig. 2. Average percentage of memory wastes in different LLM serving systems. PagedAttention drastically improves this, pushing KV cache usage efficiency to 96.3%. Adapted from Kwon *et al.* [1].

3 THE UNIFIED OS PHILOSOPHY OF AI MEMORY MANAGEMENT

Before diving into each individual system, we articulate the unifying observation that motivates this paper: the three state-of-the-art memory management architectures for LLM serving correspond, one-to-one, to three canonical Operating Systems techniques for managing the memory hierarchy.

Table 1 lays out the correspondence. The key insight is that these three systems are *complementary, not competitive*: PagedAttention solves the low-level “where to put the bytes” question, RadixAttention layers a global cache-and-sharing protocol on top, and FlexGen extends the hierarchy downward to CPU and disk when even a paged, shared cache overflows GPU VRAM. A mature production medical-AI inference stack would, in principle, compose all three: RadixAttention at the top (matching shared clinical prefixes), PagedAttention in the middle (avoiding fragmentation within the cache), and FlexGen-style offloading at the bottom (absorbing overflow into CPU RAM). Indeed, early integration work reported by the SGLang authors [2] already adopts PagedAttention as the underlying storage primitive for RadixAttention, and FlexGen [3] explicitly enumerates offloaded tree-cache management as future work.

This unified view also explains why each system’s contribution is *necessary*: dropping any layer leaves a class of waste unresolved. Without paging, a radix tree of shared prefixes still suffers internal fragmentation within each edge. Without a radix tree, paged memory still wastes bandwidth recomputing identical clinical prefixes. Without multi-tier offloading, both are constrained by the hard ceiling of GPU VRAM. The remainder of this paper makes each of these claims technically precise.

4 METHODOLOGY: THREE SYSTEMS, THREE LAYERS OF THE MEMORY HIERARCHY

4.1 PagedAttention: Virtualizing the KV Cache (vLLM)

4.1.1 Motivation and High-Level Design

The central idea of PagedAttention [1] is direct: *stop requiring KV cache tensors to be contiguous*. Just as a modern OS partitions physical RAM into pages of, say, 4 KB and permits a process’s virtual address space to map those pages in

TABLE 1
The OS-inspired correspondence that unifies the three memory management systems studied in this paper.

OS Concept	PagedAttention (vLLM)	RadixAttention (SGLang)	Multi-tier Offloading (FlexGen)
Virtual memory paging	KV blocks of B tokens, block table mapping logical→physical	Tokens grouped along radix-tree edges as KV block sequences	Activations/weights/cache paged across GPU/CPU/disk
Copy-on-write (<code>fork()</code>)	Per-block refcount; clone on write during parallel sampling / beam search	Prefix sharing via shared radix-tree edges; split on divergence	Not applicable (no forking)
Page replacement	Request-level preemption with all-or-nothing swap/recompute	LRU eviction on leaves of radix tree, respecting refcounts	Cost-model-driven eviction guided by LP solution
Hierarchical swapping	Swap KV cache to CPU RAM on preemption	Not primary focus (discussed as future work)	Core contribution: GPU ↔ CPU ↔ NVMe
I/O scheduling	Asynchronous overlap of cache transfer with compute	Asynchronous stream submission from interpreter	Zig-zag block schedule with overlap; LP-optimized
<code>exec()</code> -style code sharing	Shared-prefix prefill (system prompts)	Explicit global prefix tree across all requests	Shared weights across batches via block reuse

arbitrary order, PagedAttention partitions the KV cache into fixed-size *KV blocks* of B tokens each ($B = 16$ is the default in vLLM, chosen empirically for GPU-warp coalescing) and allows each logical sequence to map its blocks to arbitrary physical blocks.

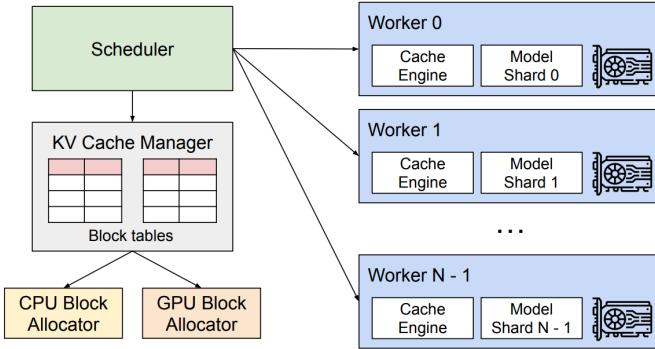


Fig. 3. vLLM system overview. A centralized block manager oversees the mapping between logical request sequences and physical GPU block tensors, while GPU workers execute model shards that consult the block table during attention computation. Adapted from Kwon *et al.* [1].

4.1.2 Formal Block-Based Attention

Concretely, let L be a sequence’s current length, B the block size, and $n_{\text{blk}} = \lceil L/B \rceil$ the number of logical blocks. The system maintains a per-sequence **block table** $\text{BT} : \{0, \dots, n_{\text{blk}} - 1\} \rightarrow \mathbb{N}$ mapping logical block indices to physical block indices in a pre-allocated GPU tensor pool $\mathcal{P} \in \mathbb{R}^{N_{\text{blk}} \times B \times d}$, where N_{blk} is the total number of physical blocks.

For a query vector q_i at logical position i , the key vector at logical position j is retrieved as

$$k_j = \mathcal{P}[\text{BT}(\lfloor j/B \rfloor)][j \bmod B], \quad (7)$$

and analogously for v_j . The attention computation of Eq. (3) is rewritten in block-wise form. Letting $K_t \in \mathbb{R}^{B \times d}$ and $V_t \in \mathbb{R}^{B \times d}$ denote the key and value tensors of the t -th physical block, we have

$$A_{i,t} = \frac{\exp(q_i K_t^\top / \sqrt{d})}{\sum_{t'=0}^{\lceil i/B \rceil - 1} \mathbf{1}^\top \exp(q_i K_{t'}^\top / \sqrt{d})}, \quad o_i = \sum_t A_{i,t} V_t. \quad (8)$$

The denominator’s sum-over-blocks is implemented as an online softmax [4] to maintain numerical stability without requiring a global intermediate. Critically, this formulation allows physical blocks to reside *anywhere* in the pool—they need not be contiguous, need not belong to any single sequence, and can be freed and reused independently.

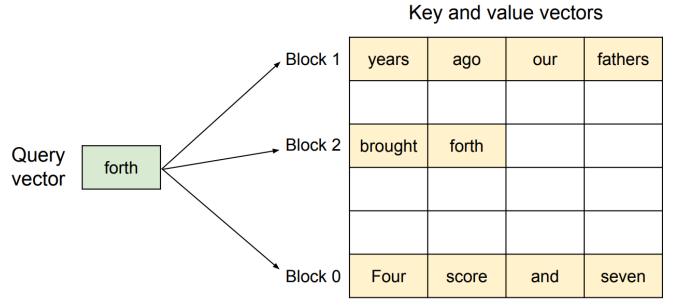


Fig. 4. PagedAttention algorithm. Key and value vectors are stored as non-contiguous blocks in memory. Each attention step fetches only the physical blocks its query actually reads, enabling flexible allocation analogous to OS physical memory frames. Adapted from Kwon *et al.* [1].

4.1.3 Block Manager Semantics

The block manager maintains three data structures: (i) a free-block bitmap over $\mathcal{P}$, (ii) the per-sequence block tables $\text{BT}^{(r)}$ for each active request r , and (iii) a per-physical-block reference counter $\text{refcnt} : \{0, \dots, N_{\text{blk}} - 1\} \rightarrow \mathbb{N}$. The reference counter is the linchpin of the **copy-on-write** mechanism described next.

When a request produces a new token at logical position j , the manager proceeds as follows:

- 1) If $j \bmod B = 0$ (a new logical block is starting), allocate a fresh physical block from the free bitmap, initialize its refcount to 1, and append its index to the block table.
- 2) Otherwise, the new (k_j, v_j) is written into the last block of the table. However, if refcnt of that physical block is > 1 (it is being shared with another sequence, e.g., a sibling in parallel sampling), the manager first performs a copy-on-write: allocate a new block, copy the existing $B - 1$ entries, decrement

the old block’s refcount, update the block table, and only then write the new token.

This mechanism generalizes to arbitrarily complex sharing topologies. The canonical case is **parallel sampling**: a user requests k samples for the same prompt. Instead of k separate prompt-phase forward passes, PagedAttention performs one prefill and then has all k sample sequences map their initial blocks to the same physical blocks with $\text{refcnt} = k$. As generation diverges, blocks are copied on first write.

4.1.4 Beam Search as a Dynamic Process Tree

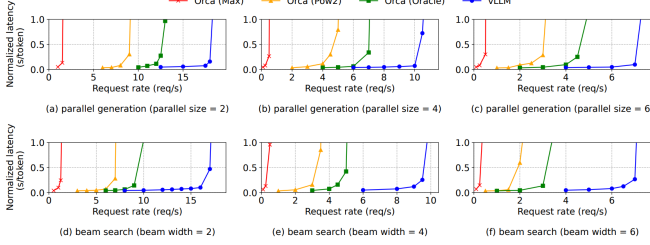


Fig. 5. Parallel generation and beam search with OPT-13B on the Alpaca dataset. Non-contiguous block-level memory management naturally accelerates complex decoding strategies by permitting fine-grained cache sharing between hypotheses. Adapted from Kwon *et al.* [1].

Beam search is where the OS-analogy becomes most vivid. In beam search with beam width k , at each decoding step the system maintains k hypotheses, expands each by all vocabulary tokens, and keeps the top- k by cumulative log-probability. Hypotheses may share ancestors for many tokens, diverge, and even have their ancestors pruned when none of their descendants survive. This is precisely the structure of an OS process tree under repeated `fork()` and `exit()`.

PagedAttention handles it elegantly: each beam hypothesis holds a block table, and shared ancestor blocks have $\text{refcnt} > 1$. When a hypothesis is pruned, its block table is released and all its blocks have their refcounts decremented; blocks whose refcount hits zero are returned to the free list, and their downstream ancestors are recursively examined. No explicit garbage collection pass is needed—the same refcounting logic that supports CoW cleans up abandoned branches. Experimentally, Kwon *et al.* report 37.6%–55.2% KV-cache savings on the Alpaca dataset at beam widths 2–6, scaling to 44.3%–66.3% on ShareGPT [1].

4.1.5 Preemption: Swap vs. Recompute

When the free-block pool is exhausted and a new token cannot be written, vLLM must *preempt* one or more sequences. The authors adopt a First-Come-First-Serve policy for admission and an “all-or-nothing” eviction rule: either all blocks belonging to a request-group are evicted together, or none are. This rule prevents partial-eviction pathologies in which one hypothesis of a beam-search request is in GPU memory while another is not.

Two recovery strategies are provided:

- **Swap-out:** Copy all evicted blocks to a CPU-side shadow pool via `cudaMemcpyAsync`, then free the

GPU blocks. On rescheduling, the blocks are copied back.

- **Recompute:** Discard the evicted blocks entirely. Concatenate the original prompt with the tokens generated so far into a single new prompt; on rescheduling, run a fresh prefill. This is surprisingly cheap because prefill enjoys full matrix-matrix batched compute, whereas the sum of all per-token decode steps the prefill replaces was matrix-vector-bound.

The trade-off depends on block size: for small blocks (1–8), recomputation wins because many tiny PCIe transfers saturate the bus inefficiently; for larger blocks (64–256), swap-out wins because bulk transfers saturate PCIe bandwidth, while recomputation cost remains roughly constant [1].

4.1.6 Distributed Execution under Tensor Parallelism

For models that do not fit on a single GPU, vLLM adopts Megatron-LM style tensor parallelism [7]: each attention head is sharded across GPUs, with an all-reduce synchronizing the per-head outputs. Crucially, all GPU workers share a single, centralized KV cache manager on the CPU. Because every worker executes the same token positions in an SPMD fashion, the logical-to-physical block table is *identical across workers*, although each worker physically stores only the portion of the block corresponding to its shard of attention heads. This single-point-of-truth design eliminates the need for distributed consensus on block allocation and dramatically simplifies correctness.

4.2 RadixAttention and Compressed FSM Decoding (SGLang)

While PagedAttention solves *where* to store KV cache, it does not address *what* to store. Medical deployments exhibit massive cross-request prefix redundancy: every diagnostic query issued to a triage agent begins with the same 2000-token hospital clinical-protocol system prompt; every chain-of-thought (CoT) few-shot prompt begins with the same set of annotated reasoning exemplars; every self-consistency sample [9] repeats the full question prefix. Re-prefilling these prefixes across thousands of daily requests is pure waste. SGLang [2] addresses this through RadixAttention, which treats the KV cache not as a collection of isolated per-request slots but as a *global, shared radix tree* indexed by token prefix.

4.2.1 The Radix Tree as a Global KV Cache

Let $\mathcal{V}$ denote the vocabulary and $\mathcal{S} = \mathcal{V}^*$ the set of all possible token sequences. A *radix tree*—a space-optimized trie in which each non-branching chain of single-child nodes is collapsed into a single edge [8]—is maintained globally across all active and recently-completed requests. Formally, $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ is a directed tree in which each edge $e \in \mathcal{E}$ is labeled with a sequence of tokens $s_e \in \mathcal{S}$. Each node $v \in \mathcal{V}$ implicitly represents the concatenation $P(v) \in \mathcal{S}$ of all edge labels on the path from the root to v ; the KV cache corresponding to $P(v)$ is stored in GPU memory (in fact, in paged form as in Section 4.1).

When a new request arrives with input sequence S_{req} , the runtime performs a *maximum prefix match*: walking from the root, at each node it follows the outgoing edge whose

label is a prefix of the remainder of S_{req} , possibly splitting an edge in the middle if only a proper prefix matches. The KV cache for the matched prefix is *reused directly*; only the unmatched suffix needs to be prefilled. Formally, if the matched prefix has length m and the full request has length n , the effective prefill length is $n - m$, and the cache hit rate for this request is m/n .

4.2.2 Complexity of the Match Operation

The time complexity of maximum prefix matching in a radix tree is

$$\mathcal{O}_{\text{match}} = \mathcal{O}(|S_{\text{req}}|), \quad (9)$$

linear in the request length. This is dramatically better than the $\mathcal{O}(|S_{\text{req}}|^2)$ cost of a full prefill pass, which for a 2000-token system prompt represents a $2000\times$ saving on the prefix portion. Even when one accounts for the attention computation dominating over the token-dispatch overhead, cache hit rates of 50%–96% reported in SGLang’s benchmarks translate directly into $2\times$ – $6.4\times$ end-to-end throughput improvements [2].

4.2.3 Reference Counting, LRU Eviction, and Dynamic Allocation

Unlike a conventional cache, which is preallocated at fixed size, the radix tree in SGLang *shares a single memory pool* with actively running requests. This design choice is motivated by an observation from the authors: in LLM serving, cache and working memory are the same kind of resource (FP16 KV tensors in paged GPU blocks), and statically partitioning them pessimizes both.

Each radix-tree node maintains a reference counter that records how many currently-running requests have that node on their active path. Nodes with $\text{refcnt} > 0$ cannot be evicted. When the system runs out of free blocks, it applies a Least-Recently-Used (LRU) policy *restricted to the evictable leaves*: the oldest leaf with $\text{refcnt} = 0$ is evicted first, potentially unlocking its parent to become a new leaf, which becomes a candidate in subsequent eviction rounds. This recursive leaf-first eviction has two attractive properties: (i) it is $\mathcal{O}(\log |V|)$ per eviction using a priority queue keyed by last-access timestamp; and (ii) it never evicts an internal node before its descendants, preserving the invariant that every cached path remains complete.

4.2.4 Cache-Aware Scheduling and the DFS Optimality Theorem

Mere caching is not enough: the order in which pending requests are admitted to the running batch dramatically affects cache hit rate. A scheduler that frequently switches between unrelated prefixes causes *cache thrashing*—loaded prefixes are evicted before their siblings arrive. SGLang addresses this with a **cache-aware scheduling** policy that, in its batch-processing mode, sorts pending requests by *matched prefix length* (longest first). This policy is equivalent, under mild conditions, to a depth-first traversal of the implicit radix tree formed by the batch.

The authors prove the following optimality result:

Theorem 1 ([2], Theorem 3.1). *For a batch of requests R whose induced radix tree is $\mathcal{T}$, an optimal cache hit rate is achieved by*

visiting $\mathcal{T}$ in depth-first-search (DFS) order, provided the cache capacity is at least the maximum request length. The longest-shared-prefix-first schedule is equivalent to such a DFS traversal.

Proof sketch. For each edge $e \in \text{edges}(\mathcal{T})$ with $|e|$ tokens, the KV cache associated with e must be computed at least once; summing gives the lower bound

$$C_{\min} = \sum_{e \in \text{edges}(\mathcal{T})} |e|. \quad (10)$$

Under DFS traversal, when edge e is first computed, the entire subtree of e is processed next; during this subtree’s computation, e is continuously referenced and—because the cache capacity exceeds the longest path—never evicted. Hence each edge is computed exactly once, attaining $C_{\min}$. The claimed equivalence of longest-shared-prefix-first order with DFS is shown by induction on the number of processed requests: at each step, the request with the longest match to already-cached content is precisely the DFS-next leaf. Full proof in [2]. $\square$

In the online (streaming) setting, the DFS order is necessarily disrupted by request arrival times, but the longest-shared-prefix heuristic remains a strong approximation, achieving on average 96% of the optimal hit rate across the SGLang benchmarks [2]. This theorem is, to our knowledge, the first formal optimality guarantee in the LLM serving literature.

4.2.5 A “Fork” Analogy

The parallel with OS memory management is especially clean here: the maximum-prefix-match operation is the AI analogue of `fork()` followed by `exec()`—it creates a new “process” (request) sharing the parent’s address space (KV cache prefix) up to the point of divergence, at which point copy-on-write semantics take over. The LRU leaf eviction is the AI analogue of page reclamation in an LRU page-replacement scheme. And the cache-aware DFS schedule is the AI analogue of co-scheduling related processes (those sharing code segments) together to improve cache locality—a classical operating-systems performance technique.

4.2.6 Compressed Finite State Machines for Structured Decoding

The second major contribution of SGLang, orthogonal to but compatible with RadixAttention, targets *constrained decoding*. In medical LLM applications, outputs must often conform to strict structural constraints: a JSON schema for a structured diagnostic report, an ICD-10 code from a controlled vocabulary, a FHIR-formatted medication order. Regular expressions suffice to describe most such constraints.

The standard technique [11] is to compile the regex into a finite state machine (FSM) whose states encode “current position in the grammar” and whose transitions are labeled with admissible next characters. At each decoding step, the model’s logits are masked to zero out tokens that would lead to an invalid state transition, then the argmax (or sample) is taken. This approach works but is grossly inefficient on one important sub-case: **constant prefix spans**. For example, in the schema `{"summary": "...", ...}`, the tokens `{, ", summary, ", :, _` are all forced by the grammar—there is

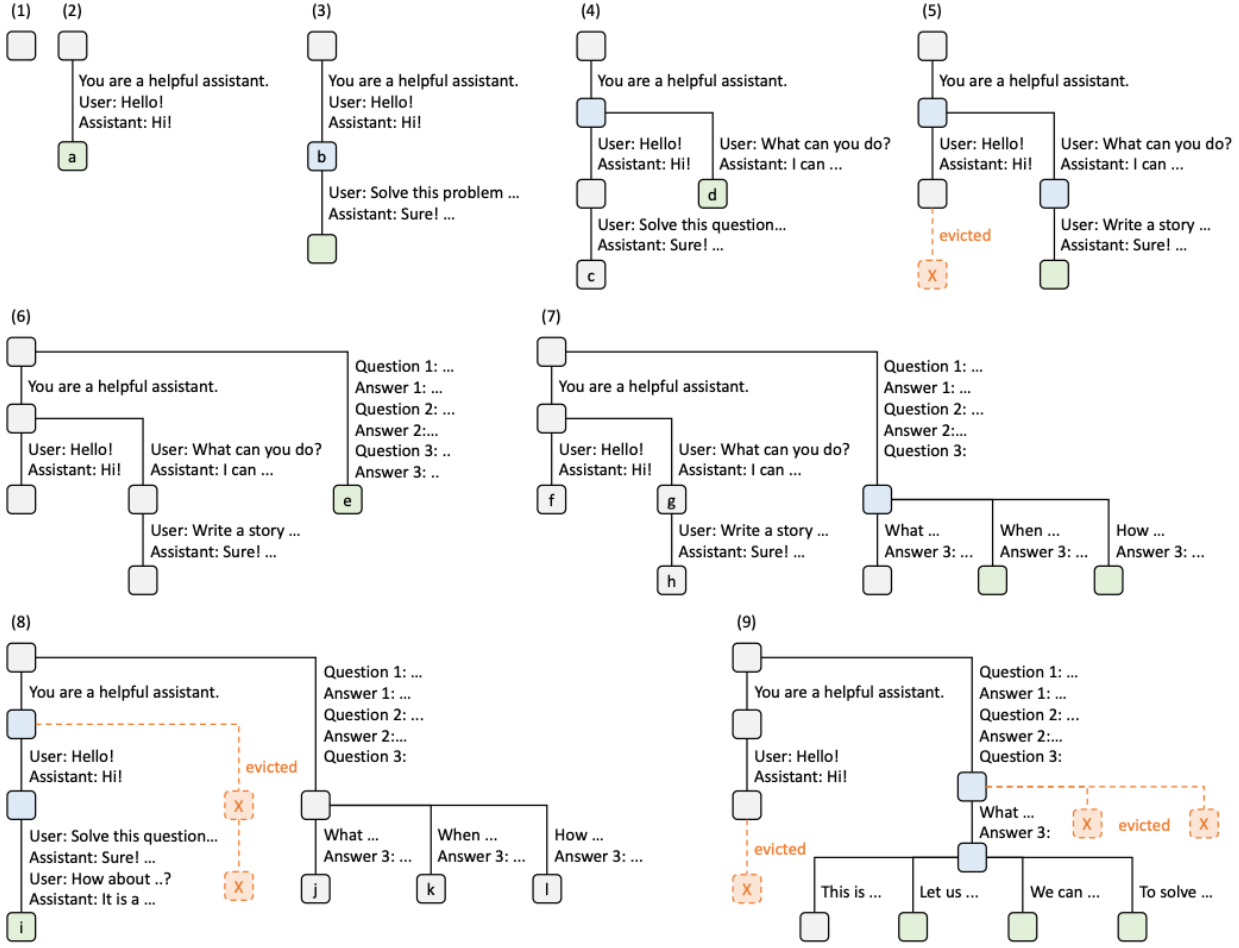


Fig. 6. Examples of RadixAttention operations with an LRU eviction policy, illustrated across nine time points. The figure demonstrates the dynamic evolution of the radix tree in response to various requests, including two chat sessions, a batch of few-shot learning inquiries, and self-consistency sampling. Each tree edge carries a label denoting a token substring. Nodes are color-coded: green for newly added, blue for cached nodes accessed during the time point, red for evicted. In step (3), a new prompt’s prefix is found and its KV cache reused. In step (4), node “b” is split so two chat sessions can share a system prompt. In step (8), least-recently-used chat-session nodes are evicted. Adapted from Zheng *et al.* [2].

no genuine sampling decision—yet the standard FSM forces the model to expend one full forward pass per token.

SGLang introduces the **compressed FSM**: the FSM construction is performed on characters (not tokens), and any chain of states $q_0 \rightarrow q_1 \rightarrow \dots \rightarrow q_k$ in which each transition has exactly one admissible character—a *singular transition chain*—is collapsed into a single edge labeled with the concatenated string. At decoding time, when the runtime enters such a compressed edge, it directly emits the entire multi-token span in a single “jump-forward” step, skipping $k - 1$ forward passes.

Two subtleties must be handled. First, **tokenization artifacts**: the concatenated string on a compressed edge must be retokenized using the model’s own tokenizer, because a naïve character-aligned split may produce token sequences that the model has never seen (e.g., splitting `summary` as `summa + ry`). SGLang resolves this by retokenizing the concatenation of all previous context and the jump string, discarding any leading tokens that already belong to the cached context. Second, **distorted probability**: if the regex offers semantically distinct alternatives that happen to lie on different compressed edges (e.g., `Excellent` vs.

`Above Average`), taking the argmax of the first-token logit may miscount the total probability of each branch. This remains an open problem, partially mitigated by prompt engineering.

Empirically, compressed-FSM decoding provides a $1.6\times$ throughput boost on JSON-schema benchmarks. For medical structured-report generation—where hundreds of tokens of JSON scaffolding surround relatively few model-chosen fields—the savings are even more pronounced.

4.3 FlexGen: Breaking the VRAM Wall via Linear-Programming-Optimized Offloading

PagedAttention and RadixAttention collectively extract nearly every useful bit out of the available GPU VRAM. What happens when even that is not enough? For a 175B-parameter model (325 GB in FP16), even an 80 GB H100 cannot load the weights, let alone serve concurrent requests. FlexGen [3] addresses this regime by treating GPU memory, CPU DRAM, and disk (NVMe SSD) as a unified three-level memory hierarchy and solving for the optimal distribution of tensors across the three levels via linear programming.

Algorithm 1 Zig-zag block schedule with I/O-compute overlap (FlexGen [3]).

```

1: for  $i = 1$  to  $n_{\text{tokens}}$  do
2:   for  $j = 1$  to  $n_{\text{layers}}$  do
3:     for  $k = 1$  to  $\text{num\_GPU\_batches}$  do
4:        $\text{LOAD\_WEIGHT}(i, j + 1, k)$   $\triangleright$  Next layer
       prefetch
5:        $\text{STORE\_ACTIVATION}(i, j, k - 1)$ 
6:        $\text{STORE\_CACHE}(i, j, k - 1)$ 
7:        $\text{LOAD\_CACHE}(i, j, k + 1)$ 
8:        $\text{LOAD\_ACTIVATION}(i, j, k + 1)$ 
9:        $\text{COMPUTE}(i, j, k)$ 
10:       $\text{SYNCHRONIZE\_ALL}$   $\triangleright$  6-way parallel sync

```

4.3.1 The Three-Level Memory Hierarchy

The physical setup FlexGen targets is a commodity workstation: 16 GB T4 GPU, 208 GB CPU DRAM, 1.5 TB NVMe SSD. Bandwidths are approximately 12 GB/s (CPU $\leftrightarrow$ GPU over PCIe 4.0 $\times$ 16) and 2 GB/s (disk $\leftrightarrow$ CPU over NVMe). Compute is asymmetric: the T4 delivers $\sim$ 65 FP16 TFLOPS for matrix multiply, while the CPU delivers roughly 0.5 TFLOPS for the same operation—but crucially, CPU compute *does not* require moving data that already lives in CPU RAM to GPU.

4.3.2 Formalizing the Search Space

Three classes of tensors must be placed in this hierarchy: model weights, activations (transient per-token intermediate tensors), and KV cache. FlexGen introduces nine continuous variables in $[0, 1]$:

wg, wc, wd : fraction of weights on GPU/CPU/Disk,
 cg, cc, cd : fraction of KV cache on GPU/CPU/Disk,
 hg, hc, hd : fraction of activations on GPU/CPU/Disk,

subject to the summation constraints $wg + wc + wd = cg + cc + cd = hg + hc + hd = 1$. Two additional integer parameters, the GPU batch size gbs and the block size bls (number of GPU batches bundled into one weight-load cycle), round out the search space.

4.3.3 Zig-Zag Block Schedule

Before optimizing the placement percentages, FlexGen must fix the *execution schedule*—the order in which the cells of the “token $\times$ layer” computation grid are visited. The naïve row-by-row schedule (finish one request’s generation before starting the next) forces a full weight reload at every row; it is manifestly I/O-bound. The column-by-column schedule (generate the t -th token of every request before moving to $t + 1$) keeps weights resident but requires storing all KV caches and activations simultaneously, which quickly overflows CPU DRAM.

FlexGen adopts a compromise: the **zig-zag block schedule**. The computation grid is partitioned into vertical blocks of bls GPU batches. Within a block, the schedule proceeds column-by-column (reusing weights); between blocks, it advances by bls rows. Algorithm 1 (adapted from [3]) gives the explicit form with overlapping.

The six I/O and compute operations are dispatched on independent CUDA streams and synchronized only at the

end of the inner body, allowing the GPU compute for batch k to overlap with the PCIe transfer for batches $k - 1$ and $k + 1$ and the disk transfer for the next weight load.

4.3.4 I/O-Optimality: The $2\times$ Bound

FlexGen’s authors prove that the zig-zag block schedule is within a factor of 2 of the I/O-optimal schedule, where I/O optimality is defined over the weight-loading cost (which dominates for large models). The optimal schedule—which they term the *diagonal block schedule* and analyze but do not implement due to practical KV-cache memory-allocation difficulties—processes the grid along diagonals.

Theorem 2 ([3], Theorem 4.1). *The I/O complexity of the zig-zag block schedule is within $2\times$ of the optimal (diagonal) block schedule.*

Proof sketch. The argument follows the “red-blue pebble game” I/O complexity framework [14]. For any valid schedule, each row of the computation grid requires at least one weight load per column; the optimal schedule achieves exactly one by arranging the diagonal so that the peak memory during a single weight load hits the CPU memory bound M' exactly. The zig-zag schedule instead has peak memory that grows linearly from $\propto s$ (at the first generated token) to $\propto s + n$ (at the last); to fit in M' , the average peak is constrained to $M'/2$, doubling the number of weight-load rounds. Hence the zig-zag schedule’s I/O cost is at most $2\times$ the optimal. The full analysis, which accounts for the identical KV-cache and activation I/O costs between the two schedules, is in [3]. $\square$

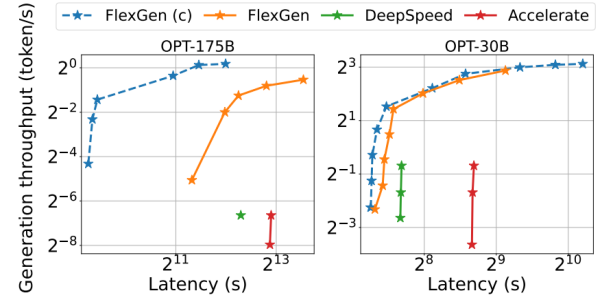


Fig. 7. Full latency and per-GPU throughput of FlexGen and Petals in different network delay and bandwidth.

4.3.5 The Full Linear Program

With the schedule fixed, the throughput-maximization problem reduces to a linear program in the nine placement variables plus the auxiliary latency estimates. Let $T = T_{\text{pre}} \cdot l + T_{\text{gen}} \cdot (n - 1) \cdot l$ be the total latency to process one block, where T_{pre} and T_{gen} are the per-layer prefill and decode latencies. Each of T_{pre} and T_{gen} is computed as a maximum over five concurrent streams:

$$T_{\text{pre}} = \max(\text{ctog}_p, \text{gtoc}_p, \text{dtoc}_p, \text{ctod}_p, \text{comp}_p), \quad (11)$$

$$T_{\text{gen}} = \max(\text{ctog}_g, \text{gtoc}_g, \text{dtoc}_g, \text{ctod}_g, \text{comp}_g). \quad (12)$$

Each of the ten terms is a linear function of the nine placement variables. For instance, the disk-to-CPU read time during generation is

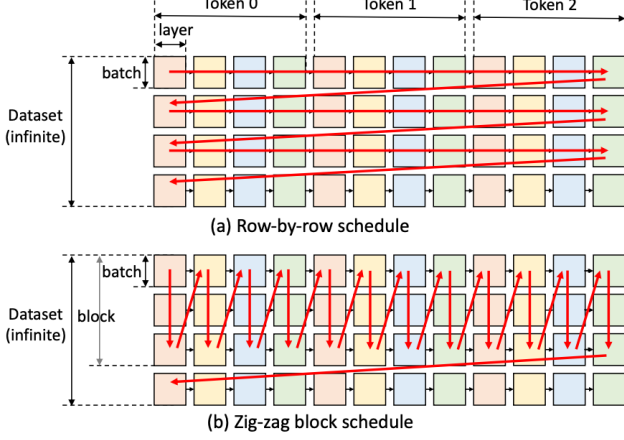


Fig. 8. Two different schedules. The red arrows denote the computation order.

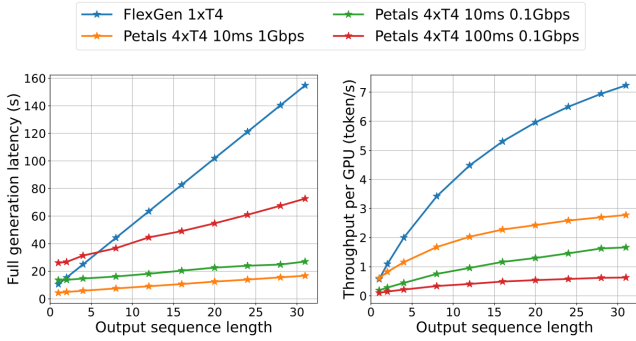


Fig. 9. Full latency and per-GPU throughput of FlexGen and Petals in different network delay and bandwidth.

$$\text{dtoc}_g = \frac{1}{\text{dtoc}_{\text{bdw}}} \left[w_d \cdot (8h_1^2 + 4h_1h_2) + 4c_d \cdot \text{bls} \cdot (s + n/2) \cdot h_1 + 2h_d \cdot h_1 \cdot \text{bls} \right]. \quad (13)$$

The full derivation (which we reproduce faithfully from [3] in the supplementary) spans roughly twenty such equations. The optimization problem is:

$$\begin{aligned} & \text{minimize}_{\mathbf{p}} && T/\text{bls} \\ & \text{subject to} && \text{gpu_peak} \leq \text{Capacity}_{\text{GPU}}, \\ & && \text{cpu_peak} \leq \text{Capacity}_{\text{CPU}}, \\ & && \text{disk_peak} \leq \text{Capacity}_{\text{Disk}}, \\ & && wg + wc + wd = 1, \quad cg + cc + cd = 1, \\ & && hg + hc + hd = 1, \\ & && 0 \leq wg, wc, wd, cg, cc, cd, hg, hc, hd \leq 1. \end{aligned} \quad (14)$$

With only nine continuous variables, this LP is solved in microseconds by off-the-shelf LP solvers (FlexGen uses CVXPY with the GLPK backend). The outer loop enumerates a small grid of (bls, gbs) candidates; the inner loop solves (14) for each candidate. The entire policy search completes in under a second even on commodity CPUs.

4.3.6 CPU-Side Attention Delegation

One further subtlety deserves attention. During decoding, the attention-score computation against the KV cache is

memory-bandwidth-bound. If the KV cache lives in CPU RAM, the naïve approach—copy the cache to the GPU, compute the attention, discard—incurs a PCIe transfer proportional to $b \cdot s \cdot h_1$. FlexGen observes that the *query* vector is only $b \cdot h_1$, a factor of s smaller than the KV cache. For $s \geq 512$, it is strictly cheaper to copy the query to the CPU, perform the matrix-vector attention on CPU, and copy the scalar output back to GPU. This is the CPU-delegation technique, and it is conditionally applied based on the LP solution.

4.3.7 Group-Wise Quantization and Sparse Attention

Two approximation techniques complement the exact offloading core. **Group-wise 4-bit quantization** [12] divides each tensor into groups of 64 contiguous elements along a chosen axis (output channel for weights, hidden dimension for KV cache), stores each element as a 4-bit integer plus per-group min/max in FP16, and dequantizes on the fly before FP16 compute. On OPT-175B, this yields a $4\times$ memory reduction with negligible accuracy loss on both Lambada [13] (next-word prediction accuracy drops from 0.758 to 0.756) and WikiText (perplexity rises from 10.82 to 10.94) [3]. Combined with offloading, 4-bit quantization often allows the entire 175B model to fit in CPU RAM, eliminating disk I/O and yielding an additional throughput jump from 0.69 to 1.12 tokens/s on a 16 GB T4.

Top- k sparse attention further exploits the empirical observation that only a small subset ($k \approx 10\%$) of the KV cache carries meaningful attention weight for a typical query. After computing the attention scores, FlexGen keeps only the top- k tokens' V-cache, reducing V-cache I/O by $10\times$. Accuracy remains within 0.1 perplexity on WikiText.

4.3.8 Extension to Multi-GPU via Pipeline Parallelism

Finally, FlexGen handles multi-GPU settings via pipeline parallelism rather than tensor parallelism. Each GPU is responsible for a contiguous range of layers; micro-batches are pipelined through the stages. Pipeline parallelism is chosen because its per-stage communication cost is constant in the number of GPUs (only the cross-stage activation is exchanged), which is ideal for the commodity interconnects FlexGen targets. With four T4 GPUs, FlexGen achieves super-linear scaling of *decoding throughput*—the relaxed per-GPU memory pressure enables a jump from disk-offloading to CPU-only offloading, which is orders of magnitude faster [3].

5 PERFORMANCE EVALUATION SYNTHESIS

This section synthesizes the published experimental results of the three systems, grouping them under the unified OS-inspired taxonomy of Section 3 and projecting them onto the medical-AI workloads of primary interest.

5.1 PagedAttention: Fragmentation Mitigation and Throughput

Fig. 2, reproduced from Kwon *et al.* [1], shows that PagedAttention reduces KV-cache memory waste from 60%–80% in baseline systems (Orca-Max, Orca-Pow2, Orca-Oracle, each representing a different over-reservation policy) to under

TABLE 2

Normalized throughput comparison for SGLang vs. baseline frameworks across complex prompting tasks on Llama-7B, A10G-24GB. Higher is better. Adapted from Zheng *et al.* [2].

Workload	SGLang	vLLM	Guidance	LMQL
MMLU (5-shot)	1.00	0.15	0.16	0.04
ReAct Agents	1.00	0.20	0.10	0.02
Generative Agents	1.00	0.31	—	—
Tree of Thought (ToT)	1.00	0.49	0.02	0.01
Skeleton of Thought	1.00	0.62	—	—
LLM Judge	1.00	0.55	—	—
JSON Decoding	1.00	0.35	0.07	0.01
Multi-Turn Chat (short)	1.00	0.54	—	—
Multi-Turn Chat (long)	1.00	0.91	—	—
DSPy RAG Pipeline	1.00	0.70	—	—

4%. The practical impact is that vLLM accommodates $2\times$ – $4\times$ larger batches at the same VRAM budget, which translates almost linearly into throughput under the memory-bound decoding regime. On Llama-7B served on a single NVIDIA A10G with the ShareGPT trace, vLLM sustains $1.7\times$ – $2.7\times$ higher request rates than Orca-Oracle (which represents an optimistic upper-bound baseline assuming perfect oracle knowledge of output lengths) at matched latency.

Fig. 5 reports beam-search throughput. For beam width 6 on the Alpaca trace, vLLM outperforms Orca-Oracle by $2.3\times$ —a gain attributable entirely to cross-hypothesis KV-cache sharing via copy-on-write, which Orca fundamentally cannot express.

5.2 RadixAttention: Prefix Reuse and Structured Decoding

Table 2 summarizes SGLang’s end-to-end throughput advantage. The gains are largest on workloads with heavy prefix sharing: 5-shot MMLU ($7\times$ over vLLM) shares the 5 examples across all queries; ReAct-agent traces ($5\times$) share the agent template and earlier-turn histories; tree-of-thought ($2\times$) shares search-tree ancestors between sibling branches. For JSON decoding, compressed FSMs layer an additional $1.6\times$ on top of the base RadixAttention gain.

Ablations (Fig. 8c of [2]) confirm that every component of RadixAttention is necessary: removing the tree structure (falling back to table-based caching), replacing cache-aware scheduling with FCFS, disabling frontend parallelism, or removing the frontend hint each independently degrades throughput by 20%–60%, and only the fully-enabled system attains peak performance. The overhead of maintaining the radix tree itself is under 0.3% on workloads with no cache-reuse opportunities—a negligible price for the dramatic upside.

5.3 FlexGen: Capacity Extension on Commodity Hardware

The headline FlexGen result is reproduced in Table 3. Running OPT-175B on a single 16GB T4—a setting that all prior systems either reject as out-of-memory or accept only at batch size 1–2—FlexGen achieves 1.12 tokens/s with 4-bit compression, a $100\times$ improvement over DeepSpeed Zero-Inference. On the HELM benchmark [15], FlexGen

TABLE 3

Generation throughput (tokens/s) of three offloading-based systems on a single NVIDIA T4 (16 GB) with 208 GB CPU DRAM and 1.5 TB SSD, for OPT-175B at input sequence length 512 and output length 32.

Adapted from Sheng *et al.* [3].

System	Max Throughput	Effective Batch
Hugging Face Accelerate	0.01	1
DeepSpeed Zero-Inference	0.01	1
Petals (decentralized)	0.08	2
FlexGen (no compression)	0.69	256
FlexGen (4-bit compression)	1.12	144

completes 7 representative sub-scenarios on OPT-30B in 21 hours of wall time on the same hardware. This is the first demonstration of which the author is aware that a 30B-class LLM can be benchmarked end-to-end, at zero cloud cost, on a single commodity GPU.

The ablation (Table 4 of [3]) confirms the primacy of the LP-based policy search. Replacing FlexGen’s LP-optimal policy with DeepSpeed’s heuristic policy degrades throughput from 7.32 to 1.57 tokens/s on OPT-30B—a $4.7\times$ regression—while holding all other system components constant. CPU compute delegation contributes a further $1.8\times$ on OPT-30B (the KV cache on OPT-175B lives largely on disk, where CPU delegation is less impactful).

5.4 Unified View: Throughput-Latency Pareto Frontier

Across all three systems, the headline observation is that each resides on, and advances, a different part of the throughput-latency Pareto frontier. PagedAttention and RadixAttention operate in the *online low-latency* regime, preserving per-token latencies below 100ms while improving throughput by 2 – $6\times$. FlexGen operates in the *offline high-throughput* regime, sacrificing per-request latency (thousands of seconds for a 175B model) in exchange for the ability to process a 144-request batch that would otherwise be impossible. For medical AI deployments, both regimes matter: real-time triage agents demand the former, while overnight cohort-wide report generation favors the latter.

6 CASE STUDY: APPLICATION TO MEDICAL AI DEPLOYMENT

We now discuss how each of the three techniques maps onto four representative medical-AI inference workloads, drawing on recent trends in dermatological AI, biomedical-agent systems, and regulated clinical deployment.

6.1 Dermoscopic Foundation Model Serving

Dermoscopy foundation models [16] encode high-resolution (1024×1024) skin-lesion images into patch-token sequences of length 576–2048, which are then either classified directly or fed into a VLM alongside free-text clinical metadata. A single lesion consultation may involve several images per patient (ABCDE views, polarized vs. non-polarized light, time-series for change monitoring), yielding 2000–8000 image tokens plus 500–2000 text tokens per request.

PagedAttention is essential here: the extreme variability of request length (a naive pediatric case may be 500 tokens,

a complex melanoma workup may be 8000) would cause crippling internal fragmentation under contiguous allocation. RadixAttention is equally valuable: every consultation prepends the same hospital-wide dermatology system prompt (typically 1500–2500 tokens of clinical guidelines) and often references shared atlas examples; radix-tree prefix matching recovers this at the tree level. For a large teaching hospital processing 2000 dermoscopy consultations per day, the compounded savings are substantial: if each consultation shares an average 2000-token prefix, RadixAttention saves approximately $2000 \times 2000 = 4 \times 10^6$ token-equivalents of prefill computation daily.

6.2 Biomedical-Agent Frameworks (Biomni-Style)

Recent biomedical-agent systems orchestrate tool-use across databases (PubMed, UniProt, ChEMBL), structured knowledge bases (Gene Ontology), and specialized models (AlphaFold, ESMFold) via ReAct-style [10] prompting. A typical agent trajectory comprises 4–12 tool-calling turns, each of which re-serializes the full conversation-plus-tool-output history into the next LLM call. The prefix redundancy is thus extreme: turn t ’s input is turn $t-1$ ’s input concatenated with turn $t-1$ ’s output. Without RadixAttention, this is an $\mathcal{O}(T^2)$ prefill workload in the number of turns T ; with RadixAttention, it is $\mathcal{O}(T)$. For the Biomni framework specifically, production traces would likely exhibit >90% cache hit rates under RadixAttention, translating into $5\times$ – $10\times$ end-to-end latency reductions.

The compressed FSM is also directly applicable: biomedical agents emit structured tool-invocation JSON, and the constant scaffolding (`{"tool": "`, `"", "args": {`, etc.) represents 30–60% of emitted tokens. Compressed FSMs recover this constant fraction at nearly zero decoding cost.

6.3 Structured Clinical Report Generation

Regulatory environments (FDA, EMA, China’s NMPA) increasingly favor structured clinical outputs—SNOMED-CT concepts, ICD-10-CM codes, HL7 FHIR resources—over free text, because the former are auditable, queryable, and interoperable. Generating such outputs from an LMM is precisely the constrained-decoding setting. A radiology-report schema may contain 40–80 constrained fields (findings, impressions, recommendations, priority, each drawn from controlled vocabularies); JSON scaffolding and field names consume $\approx 60\%$ of emitted tokens, all on compressed FSM edges. The net effect is that structured report generation under SGLang’s compressed FSM decodes at a rate comparable to *unconstrained* free-text generation, removing a major historical argument against structured outputs.

6.4 On-Premise Deployment and Data Privacy

Protected Health Information (PHI) cannot, in many jurisdictions, leave the hospital network. On-premise deployment is therefore mandatory for all but the most stripped-down applications, but hospitals rarely possess multi-A100 GPU clusters. FlexGen directly addresses this gap: a 30B-class model can be deployed on a single consumer-grade workstation (RTX 4090 + 256GB DDR5 + fast NVMe) for overnight batch processing of the day’s cohort, with total

latency on the order of hours and zero PHI egress. For some use cases (e.g., morning-rounds triage of the prior night’s ER admissions), this latency is acceptable; for others (real-time imaging triage), PagedAttention + RadixAttention on a shared on-premise A100 remains preferable. The choice between the two regimes is a deployment-engineering decision, informed by the LP-optimal policy FlexGen provides for any given hardware.

6.5 Composition: A Hypothetical Full-Stack Medical Serving Architecture

Although no production system integrates all three techniques end-to-end as of this writing, the OS-inspired framework of Section 3 suggests an obvious composition:

- 1) The top-level scheduler maintains a RadixAttention tree over all active and recently-completed requests. Incoming requests are matched against the tree; prefix hits skip prefill.
- 2) Each radix-tree node’s KV cache is stored, internally, in PagedAttention blocks. This permits fine-grained allocation without fragmentation and naturally supports the copy-on-write semantics needed when two radix-tree branches diverge.
- 3) When the GPU block pool is exhausted, an LP similar to FlexGen’s (but over only two tiers—GPU and CPU—and over only KV-cache placement) determines which radix-tree subtrees to evict to CPU. This is a natural generalization of vLLM’s swap-out logic.
- 4) Constrained-output endpoints compile their regexes to compressed FSMs in advance and dispatch through a specialized decode kernel.

We conjecture that such a composed system would yield multiplicative, not merely additive, speedups over any single technique.

7 DISCUSSION AND OPEN PROBLEMS

7.1 User-Space Emulation vs. Kernel-Level Integration

A striking feature of all three systems is that they emulate OS primitives at the *application layer*. PagedAttention implements its own block allocator, its own reference counter, and its own page table, entirely in Python and CUDA kernels; RadixAttention maintains its radix tree in CPU-side Python; FlexGen’s LP is solved by a userspace solver on the application CPU. This is an engineering pragmatism—bypassing the OS and CUDA driver lets researchers iterate quickly—but it imposes real costs. Every PagedAttention kernel call must traverse the block table via a pointer indirection that a true MMU would handle in hardware; every radix-tree update incurs a Python-level lock acquisition; every LP solve duplicates work that a kernel-level swap daemon could amortize.

The next frontier is therefore **pushing these primitives down the stack**. We identify three candidate paths. (i) **Kernel-bypass via eBPF**: offload the KV-cache scheduling policy into an eBPF program attached to a custom CUDA driver, eliminating Python overhead. (ii) **Hardware-assisted paged attention**: exploit the Hopper architecture’s Tensor

Memory Accelerator and distributed shared memory to implement a hardware-managed KV block table, analogous to an MMU. (iii) **OS-level LLM-aware swapping**: extend the Linux page cache with LLM-aware heuristics (prefer evicting pages last touched by the beam-search worst hypothesis, etc.).

7.2 Limitations of the Three Systems

Each system has known limitations worth noting:

- **PagedAttention** introduces a 20–26% overhead on the attention kernel itself due to block-table indirection, though this is amortized over the entire transformer forward pass and contributes <5% to end-to-end latency.
- **RadixAttention**, in its current form, can suffer from starvation: long-tail requests whose prefix never matches anything may be indefinitely postponed in favor of cache-hit-heavy requests. The authors acknowledge this and cite fair-scheduling extensions [17] as future work. Moreover, compressed-FSM decoding handles distorted probabilities imperfectly, as discussed in Section 4.2.6.
- **FlexGen** targets throughput, not latency; in interactive medical-decision-support scenarios its per-request latencies (often thousands of seconds for 175B models) are unacceptable. Moreover, its LP ignores several real-world effects—PCIe contention under concurrent use, thermal throttling, NUMA effects on multi-socket servers—which can degrade the achieved throughput by 10–30% relative to the LP’s prediction.

7.3 Open Research Problems

Several open problems follow naturally from the preceding analysis:

- 1) **Fuzzy-prefix matching in RadixAttention.** Current matching is exact: two prompts that differ in a single whitespace or synonym produce zero cache hit. For clinical free text, where trivial differences are pervasive, an embedding-based fuzzy prefix matcher (with bounded accuracy degradation) could raise cache hit rates dramatically.
- 2) **Three-tier RadixAttention.** Extending the radix tree to span GPU, CPU, and disk (as FlexGen does for raw tensors) would allow massive historical-trace caches without bloating GPU VRAM. The main obstacle is the design of a cost-aware eviction policy across three tiers.
- 3) **Co-optimization of tokenizer and compressed FSM.** Current compressed FSM suffers retokenization overhead because tokenizers are oblivious to the grammar. A grammar-aware tokenizer, trained to align token boundaries with grammar transitions, could eliminate retokenization entirely.
- 4) **LP generalization to heterogeneous hardware.** FlexGen’s LP assumes a fixed hardware topology. Real-world deployments span heterogeneous fleets (mixed T4 + A100 + MI300). An online LP that resolves as the fleet’s load shifts would enable elastic medical-AI serving.

7.4 Implications for the Medical-AI Community

For researchers and engineers building clinical LLM/LMM systems, the concrete takeaways are: (i) never deploy an LMM on a naive serving stack—the memory waste is prohibitive and the QPS ceiling is artificially low; (ii) RadixAttention-style prefix caching is a near-free $2\times\text{--}5\times$ throughput boost on any workload with shared system prompts, which is all clinical workloads; (iii) structured-output generation is no longer a performance tax thanks to compressed FSMs—prefer structured outputs for auditability; (iv) on-premise deployment of 30B+ models is feasible on consumer hardware via FlexGen-style offloading, eliminating the false trade-off between model quality and PHI compliance.

8 CONCLUSIONS

The transition from primitive LLM serving to modern, production-grade AI memory management has, in retrospect, been a story of *rediscovering Operating Systems wisdom*. PagedAttention is virtual-memory paging applied to the KV cache. RadixAttention is copy-on-write `fork()` plus LRU page replacement applied at the granularity of prompt prefixes. FlexGen is hierarchical swapping plus I/O scheduling applied to the tensor placement problem, formalized as a linear program and proved to be within $2\times$ of I/O-optimality. Each technique addresses a distinct source of inefficiency—fragmentation, redundancy, and hard VRAM ceilings, respectively—and their composition would, in principle, yield a system that dominates each individually.

For medical artificial intelligence specifically, these techniques are not mere engineering optimizations but the *enabling substrate* for clinically credible deployment. High-resolution diagnostic imagery, long patient histories, multi-turn agent reasoning, structured regulatory outputs, and on-premise PHI compliance each place demands on the serving stack that would be unmet by any naive implementation. The three systems studied here, taken together, render those demands tractable on commodity hardware.

A current limitation, which we flag as the clearest target for future research, is that all three systems emulate their OS primitives purely at the application layer. Pushing block management, prefix sharing, and tier-aware swapping into the CUDA driver—or, more ambitiously, into a custom Linux kernel path via eBPF—would eliminate tens of percent of current overhead and open the door to true zero-copy, hardware-MMU-assisted attention. We believe this hybrid systems-ML research direction will define the next decade of efficient LMM inference.

ACKNOWLEDGMENT

The author thanks his thesis advisors and the Chinese University of Hong Kong, Shenzhen, for supporting the medical AI research that motivated this survey.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

- [2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," *arXiv:2312.07104v2*, Jun. 2024.
- [3] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proc. 40th International Conference on Machine Learning (ICML)*, vol. 202, 2023, pp. 31094–31116.
- [4] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16344–16359.
- [5] NVIDIA Corp., "FasterTransformer," <https://github.com/NVIDIA/FasterTransformer>, accessed 2024.
- [6] Hugging Face, "Accelerate," <https://huggingface.co/docs/accelerate/index>, 2022.
- [7] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv:1909.08053*, 2019.
- [8] D. R. Morrison, "PATRICIA—Practical Algorithm to Retrieve Information Coded in Alphanumeric," *Journal of the ACM*, vol. 15, no. 4, pp. 514–534, 1968.
- [9] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proc. ICLR*, 2023.
- [10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.
- [11] B. T. Willard and R. Louf, "Efficient Guided Generation for Large Language Models," *arXiv:2307.09702*, 2023.
- [12] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT," in *Proc. AAAI*, vol. 34, no. 5, 2020, pp. 8815–8821.
- [13] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández, "The LAMBADA Dataset: Word Prediction Requiring a Broad Discourse Context," in *Proc. ACL*, 2016, pp. 1525–1534.
- [14] H. Jia-Wei and H.-T. Kung, "I/O Complexity: The Red-Blue Pebble Game," in *Proc. 13th Annual ACM Symposium on Theory of Computing*, 1981, pp. 326–333.
- [15] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, et al., "Holistic Evaluation of Language Models," *arXiv:2211.09110*, 2022.
- [16] S. Yan, Z. Yu, C. Primiero, C. Vico-Alonso, Z. Wang, L. Yang, P. Tschandl, M. Hu, L. Ju, G. Tan, V. Tran, A. Chen, R. Xiao, J. Ren, H. Wang, L. Wang, A. K. Pandey, A. Soyer, B. Swain, H. Kittler, M. Janda, P. Soyer, and Z. Ge, "A General-Purpose Multimodal Foundation Model for Dermatology," *arXiv:2410.15038*, 2024.
- [17] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, "Fairness in Serving Large Language Models," *arXiv:2401.00588*, 2023.
- [18] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in *Proc. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.
- [19] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, et al., "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *Proc. SC22*, 2022, pp. 646–660.
- [20] A. Borzunov, D. Baranchuk, T. Dettmers, M. Ryabinin, Y. Belkada, A. Chumachenko, P. Samygin, and C. Raffel, "Petals: Collaborative Inference and Fine-Tuning of Large Models," *arXiv:2209.01188*, 2022.
- [21] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers," *arXiv:2210.17323*, 2022.
- [22] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Advances in Neural Information Processing Systems*, 2022.
- [23] G. Xiao, J. Lin, M. Seznec, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," *arXiv:2211.10438*, 2022.
- [24] Z. Ye, L. Chen, R. Lai, Y. Zhao, S. Zheng, J. Shao, B. Hou, H. Jin, Y. Zuo, L. Yin, T. Chen, and L. Ceze, "Accelerating Self-Attentions for LLM Serving with FlashInfer," Feb. 2024.
- [25] P. Tillet, H.-T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proc. MAPL*, 2019, pp. 10–19.
- [26] H. Touvron, L. Martin, K. Stone, P. Albert, et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv:2307.09288*, 2023.
- [27] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, et al., "OPT: Open Pre-trained Transformer Language Models," *arXiv:2205.01068*, 2022.
- [28] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," *arXiv:2310.03714*, 2023.
- [29] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," *arXiv:2305.10601*, 2023.
- [30] (Biomedical Agent Framework), Representative works such as R. Zhao et al., "BioMed-Agent: A Foundation-Model-Driven Autonomous Agent for Biomedical Research," and follow-up systems in 2024–2026.
- [31] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, et al. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. *arXiv preprint arXiv:2207.00032* (2022).
- [32] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [33] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. *Advances in neural information processing systems* 13 (2000).
- [34] Ondrej Bojar, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Varvara Logacheva, Christof Monz, Matteo Negri, Aurelie Neveol, Mariana Neves, Martin Popel, Matt Post, Raphael Rubino, Carolina Scarton, Lucia Specia, Marco Turchi, Karin Verspoor, and Marcos Zampieri. 2016. Findings of the 2016 Conference on Machine Translation. In *Proceedings of the First Conference on Machine Translation*.
- [35] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [36] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [37] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [38] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- [39] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311* (2022).
- [40] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. 2020. InferLine: latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, 477–491.
- [41] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 613–627.
- [42] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. 2022. DVABatch: Diversity-aware Multi-Entry Multi-Exit Batching for Efficient Processing of DNN Services

- on GPUs. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 183–198.
- [43] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
 - [44] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. TurboTransformers: an efficient GPU serving system for transformer models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 389–402.
 - [45] FastAPI. 2023. FastAPI. <https://github.com/tiangolo/fastapi>.
 - [46] Pin Gao, Lingfan Yu, Yongwei Wu, and Jinyang Li. 2018. Low latency rnn inference with cellular batching. In *Proceedings of the Thirteenth EuroSys Conference*. 1–15.
 - [47] Amir Gholami, Zhewei Yao, Sehoon Kim, Michael W Mahoney, and Kurt Keutzer. 2021. Ai and memory wall. *RiseLab Medium Post* 1 (2021), 6.
 - [48] Github. 2022. <https://github.com/features/copilot>
 - [49] Google. 2023. <https://bard.google.com/>
 - [50] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 443–462.
 - [51] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 539–558.
 - [52] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
 - [53] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.
 - [54] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems* 2 (2020), 497–511.
 - [55] Tom Kilburn, David BG Edwards, Michael J Lanigan, and Frank H Sumner. 1962. One-level storage system. *IRE Transactions on Electronic Computers* 2 (1962), 223–235.
 - [56] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).
 - [57] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).
 - [58] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. *arXiv preprint arXiv:2302.11665* (2023).
 - [59] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling holistic deep learning compiler optimizations with rtasks. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. 881–897.
 - [60] NVIDIA. [n. d.]. Triton Inference Server. <https://developer.nvidia.com/nvidia-triton-inference-server>.
 - [61] NVIDIA. 2023. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>.
 - [62] NVIDIA. 2023. NCCL: The NVIDIA Collective Communication Library. <https://developer.nvidia.com/nccl>.
 - [63] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139* (2017).
 - [64] OpenAI. 2020. <https://openai.com/blog/openai-api>
 - [65] OpenAI. 2022. <https://openai.com/blog/chatgpt>
 - [66] OpenAI. 2023. <https://openai.com/blog/custom-instructions-forchatgpt>
 - [67] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774* [cs.CL]
 - [68] LMSYS ORG. 2023. Chatbot Arena Leaderboard Week 8: Introducing MT-Bench and Vicuna-33B. <https://lmsys.org/blog/2023-06-22-leaderboard/>.
 - [69] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
 - [70] Shishir G Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. 2022. POET: Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging. In *International Conference on Machine Learning*. PMLR, 17573–17583.
 - [71] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. Efficiently Scaling Transformer Inference. *arXiv preprint arXiv:2211.05102* (2022).
 - [72] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training.. In *USENIX Annual Technical Conference*. 551–564.
 - [73] Reuters. 2023. <https://www.reuters.com/technology/tech-giants-ai-like-bing-bard-poses-billion-dollar-search-problem-2023-02-22/>
 - [74] Amazon Web Services. 2023. <https://aws.amazon.com/bedrock/>
 - [75] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. Nexus: A GPU cluster engine for accelerating DNN-based video analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 322–337.
 - [76] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E Gonzalez, et al. 2023. High-throughput Generative Inference of Large Language Models with a Single GPU. *arXiv preprint arXiv:2303.06865* (2023).
 - [77] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-Lm: Training multibillion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
 - [78] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. 2022. OLLA: Optimizing the Lifetime and Location of Arrays to Reduce the Memory Usage of Neural Networks. (2022). <https://doi.org/10.48550/arXiv.2210.12924>
 - [79] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. *Advances in neural information processing systems* 27 (2014).
 - [80] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
 - [81] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
 - [82] Jing Wang, Youyou Lu, Qing Wang, Minhui Xie, Keji Huang, and Jiwei Shu. 2022. Pacman: An Efficient Compaction Approach for Log-Structured Key-Value Store on Persistent Memory. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 773–788.
 - [83] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*. 41–53.
 - [84] Xiaohui Wang, Ying Xiong, Yang Wei, Mingxuan Wang, and Lei Li. 2021. LightSeq: A High Performance Inference Library for Transformers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers*. 113–120.
 - [85] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-Instruct: Aligning Language Model with Self Generated Instructions. *arXiv preprint arXiv:2212.10560* (2022).
 - [86] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.

- [87] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144 (2016).
- [88] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 521–538.
- [89] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). USENIX Association, Boston, MA, 787–808.
- [90] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. arXiv preprint arXiv:2205.01068 (2022).
- [91] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 559–578.
- [92] Zhe Zhou, Xuechao Wei, Jiejing Zhang, and Guangyu Sun. 2022. PetS: A Unified Framework for Parameter-Efficient Transformers Serving. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 489–504.